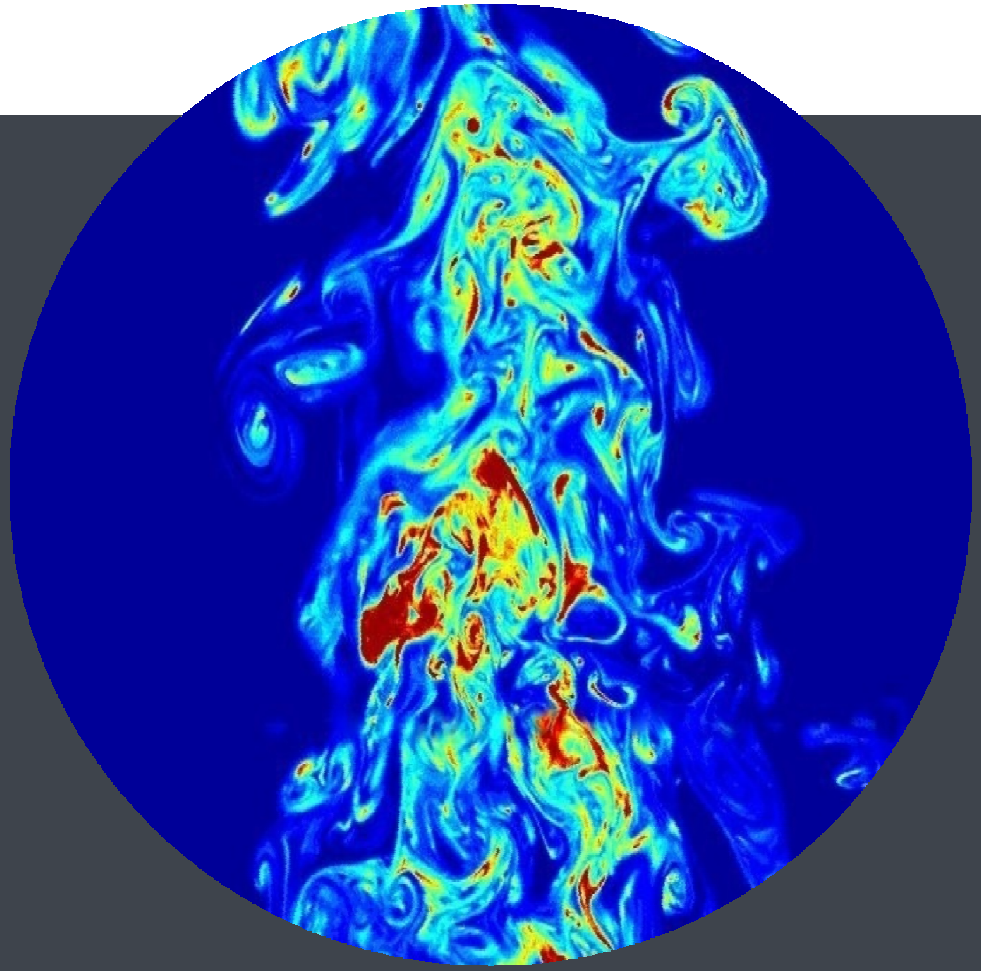
Universität Stuttgart
Institute of Combustion Technology

Modelling particle flame synthesis with mmcFoam

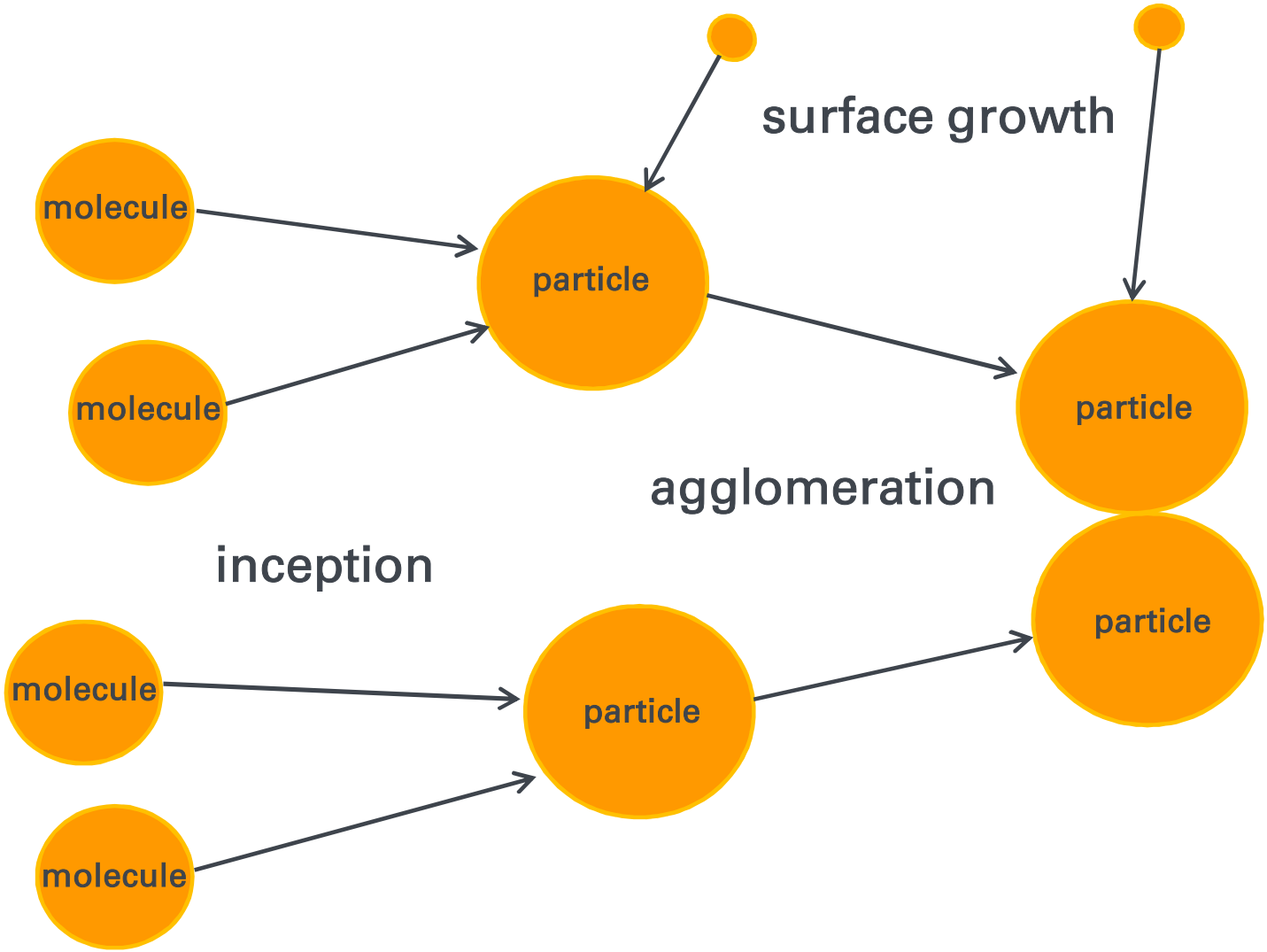


Gregor Neuber

2018 / 03 / 20

Theoretical background

Differentiation of the physical processes



Theoretical background

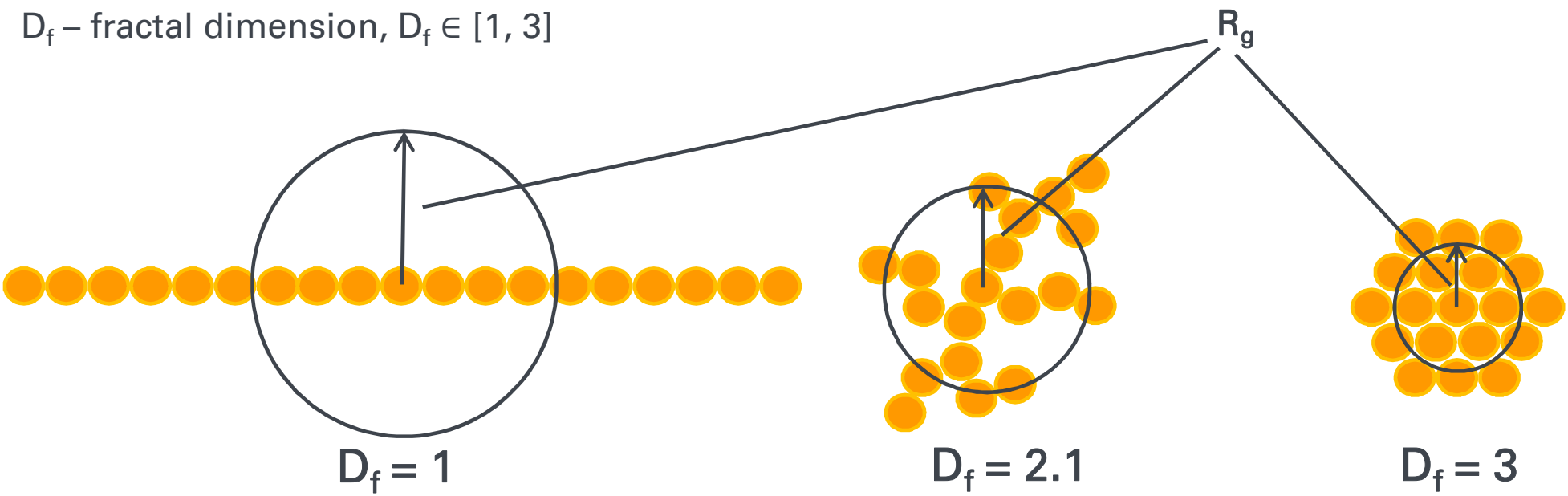
Power law of agglomerates

For solid particles of arbitrary shape the following model can be applied:

n_p – number of primary particles
v – particle volume
v_0 – volume of primary particle
k_f – fractal pre-factor, $k_f \in [0.7, 2.4]$
R_g – radius of gyration
d_p – diameter of primary particle
D_f – fractal dimension, $D_f \in [1, 3]$

input parameter

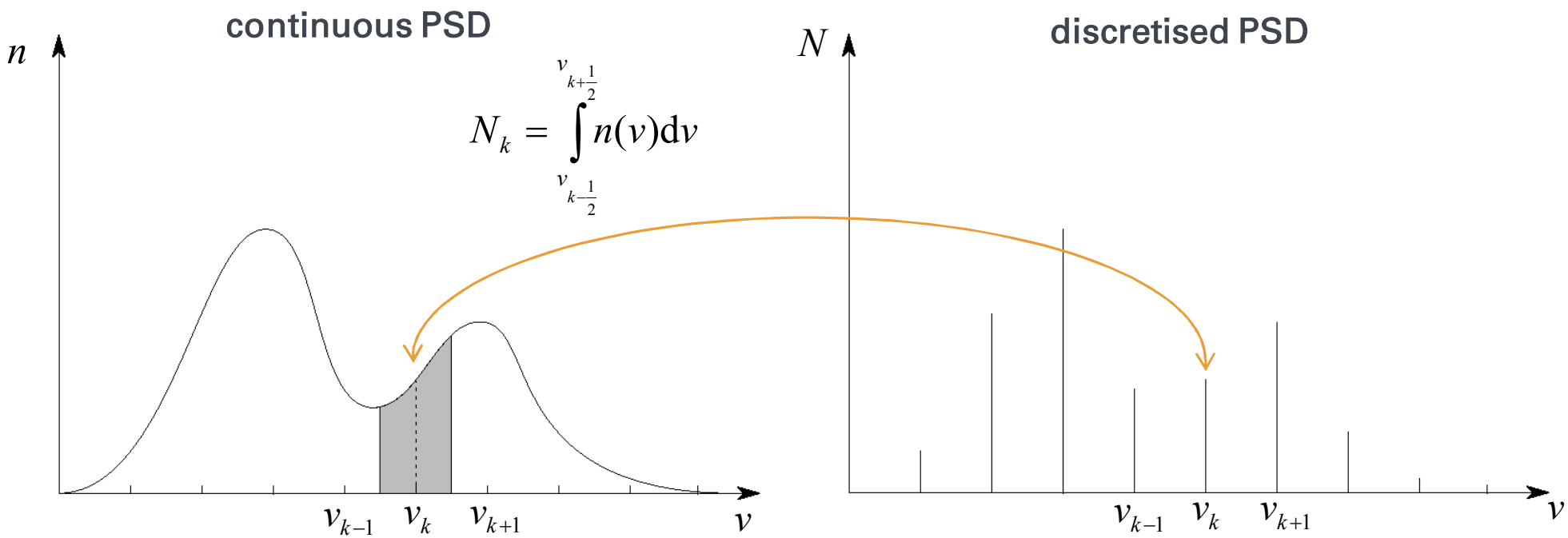
$$n_p = \frac{v}{\boxed{v_0}} = \boxed{k_f}\left(\frac{R_g}{\boxed{d_p}/2}\right)^{\boxed{D_f}}, \quad n_p > 1000$$



Theoretical background

Nodal form of the sectional approach

A very efficient way to represent the particle size distribution is the nodal form of the sectional method where the continuous PSD is discretised into so called sections by integrating continuous PSD in size space.



Theoretical background

Transport equation of the particle number density

In our model the Particle Size Distribution (PSD) is modelled by the discretised version of the Population Balance Equation (PBE):

$$\frac{\partial N_k}{\partial t} + \frac{\partial}{\partial x_j}\left(u_j N_k - D_k \frac{\partial N_k}{\partial x_k} \right) = \boxed{\dot{s}_{k,inc} + \dot{s}_{k,surf} + \dot{s}_{k,agg}}$$

For mmcFoam the particle number densities, N_k, are additional transported scalars on the stochastic particles used in mmcFoam. The convection of the number densities are described by the displacement of these stochastic particles. The (turbulent) diffusion is modelled by the MMC mixing model.

The source terms account for:

- solid particle inception,
- surface growth and
- agglomeration

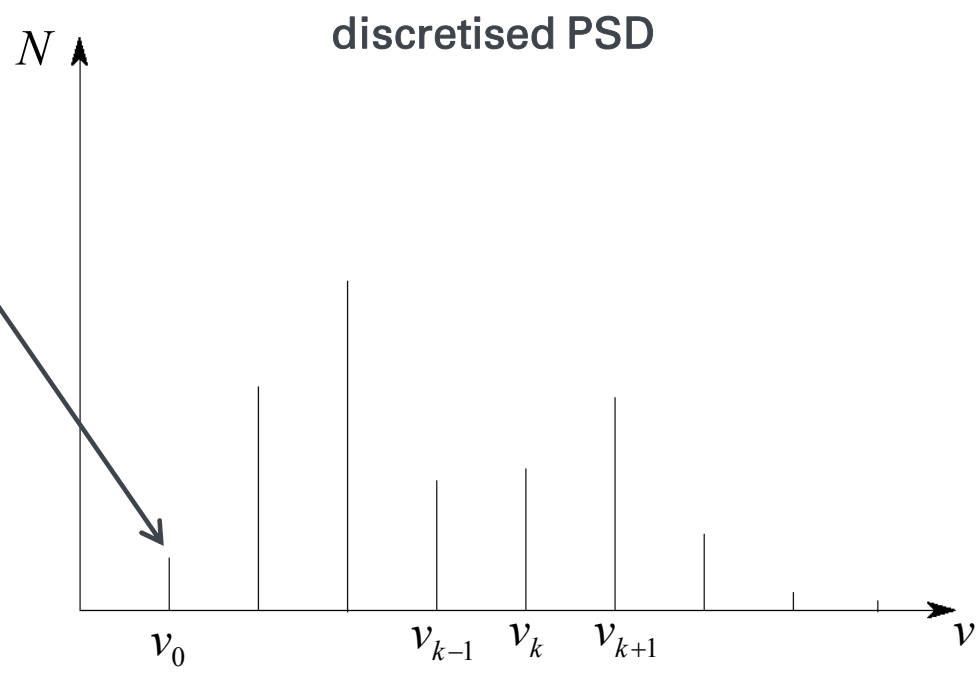
and models are needed to close the PBE equation on the source term level.

Implementation of the synthesis model in mmcFoam

Solid particle inception

The inception rate of (mean) primary particles is given by the production rate of precursor species described by a finite rate chemistry model:

$$\dot{s}_{k=0,inc} = \frac{Y_{precursor}}{\Delta t} \frac{\overline{M} p N_A}{M_{precursor} R_u T}$$



$\dot{S}_{k=0}$ – inception rate for zeroth section
Y – mass fraction
T – temperature
N_A – Avogadro constant
p – pressure
M – molecular weight
R_u – universal gas constant
Δt – time step

By assuming spherical particles the diameter of the primary particles are assumed to be d_0 = 0.98 nm.

Implementation of the synthesis model in mmcFoam

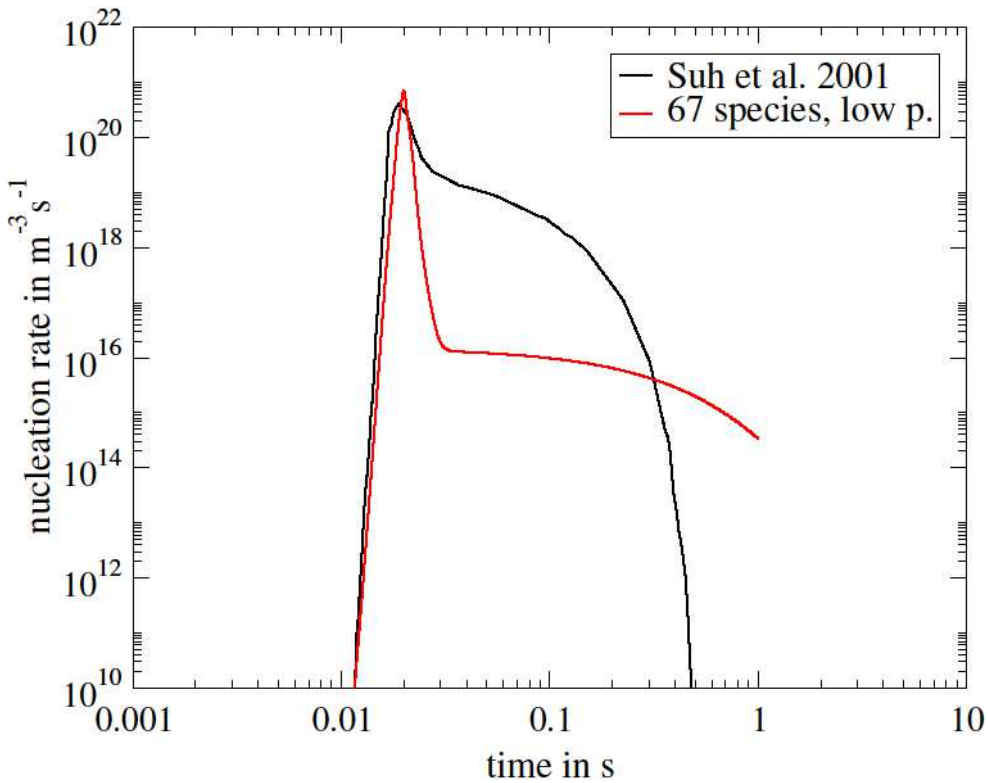
Solid particle inception

```
template <class CloudType>
void Foam::SolidParticleFlameSynthesis<CloudType>::particleInception(particleType& particle)
{
    CloudType& cloud(this->owner());
    const scalar deltaT = cloud.mesh().time().deltaT().value(); // [s]

    const scalar molecularWeightMixture = calcMolecularWeightMixture(particle.Y());   // [kg/kmol]

    particle.sectionSource() = 0.0;

    forAll(speciesThermo_, speciesI)
    {
        const word speciesName = this->owner().thermo().carrier().species()[speciesI];

        if (precursorSpeciesDict_.found(speciesName))
        {
            precursorSpeciesIndex_ = speciesI;

            const scalar molecularWeightPrecursorSpecies = speciesThermo_[precursorSpeciesIndex_].W(); // [kg/kmol]

            // calculate the number of particles inserted into first section due to formation of precursor species
            const scalar insertedParticles = NA_*particle.Y()
            [precursorSpeciesIndex_]*particle.pc()*molecularWeightMixture/(Ru_*particle.T()*molecularWeightPrecursorSpecies); //
            [#/m^3] = [1/kmol] [1] [kg/m s^2] [kg/kmol] / [kg m^2/s^2 kmol K] [K] [kg/kmol]

            particle.sections()[0] += insertedParticles;

            // calculate the nucleation rate
            particle.sectionSource() += insertedParticles/deltaT; // [#/m^3 s]

            particle.Y()[solidSpeciesIndex_]    += particle.Y()[precursorSpeciesIndex_];
            particle.Y()[precursorSpeciesIndex_] = 0.0;
        }
    }
}
```

Implementation of the synthesis model in mmcFoam

Solid particle inception

Validation of the particle inception method can be done by comparing our simulation results with results obtained with other codes. Here I compare the inception/nucleation rate with the incpetion rate obtained within a moment method approach presented in Suh et al. (2001).



This numerical experiment tests the nucleation of particles at p = 107 Pa and T = 773 K for a oxygen to silane ratio of R = 15. The underlying mechanism is presented in Suh et al. (2001) utilising 67 species and 264 reactions.

It is assumed that the apparent differences are due to different thermodynamic data (NASA coefficients).

The maximum and the trend is generally well matched.

Implementation of the synthesis model in mmcFoam

Surface growth of agglomerates

A molecule from the gas-phase which collides with a particle sticks on the particle and increases the mass of the particle. This process is implemented in the model assuming that the collision of a molecule from the gas-phase and a particle can be calculated using the free molecular collision kernel for the particle and a single molecule from the gas phase. The rate takes the form:

$$R_{surf}^{fm} = 2.2\eta M_{dep} C_{dep} \sqrt{\frac{\pi k_B T}{2m}} \left(d + d_c\right)^2.$$

Where C, m and d are the gas-phase concentration, mass, and diameter of the condensing species molecule, respectively. d_c is the collision diameter of the particle and η the efficiency of the collision. The growth rate is given in kg/s.

If the particle is very large another process becomes important because the surface growth is determined by how many molecules diffuse towards the particles' surface. This rate can be given by the analysis of the near flow field and the model are well known as the diffusion limited growth model for the continuum regime:

$$R_{surf}^{cont} = 2\rho_{dep} D_{dep} \frac{Y_{dep}}{d/2} A.$$

Implementation of the synthesis model in mmcFoam

Surface growth of agglomerates

```
template <class CloudType>
void Foam::SolidParticleFlameSynthesis<CloudType>::particleSurfaceGrowth(particleType& particle)
{
    CloudType& cloud(this->owner());
    const scalar deltaT = cloud.mesh().time().deltaT().value(); // [s]

    const scalar molecularWeightMixture = calcMolecularWeightMixture(particle.Y()); // [kg/kmol]

    const scalar densityMixture = particle.pc()*molecularWeightMixture/(Ru_*particle.T()); // [kg/m^3] = [kg/m s^2] [kg/kmol] / [kg m^2/s^2 kmol K] [K]

    forAll(speciesThermo_, speciesI)
    {
        const word speciesName = this->owner().thermo().carrier().species()[speciesI];

        if (depositingSpeciesDict_.found(speciesName))
        {
            depositionSpeciesIndex_ = speciesI;

            scalarList sectionChange(particle.nSections(), 0.0);

            const scalar molecularWeightDepositingSpecies = speciesThermo_[depositionSpeciesIndex_].W();                    // [kg/kmol]

            const scalar monomerMassDepositingSpecies     = molecularWeightDepositingSpecies/NA_;                          // [kg/#] = [kg/kmol] / [#/kmol]
            const scalar monomerVolumeDepositingSpecies   = monomerMassDepositingSpecies/solidDensity_;                    // [m^3/#] = [kg/#] / [kg/m^3]

            forAll (particle.sections(), section)
            {
                // calculation of the volumetric growth via the free molecular collision kernel approach
                const scalar d                    = Foam::pow(6.0*monomerVolumeDepositingSpecies/M_PI,1.0/3.0);              // [m]

                const scalar gasPhaseConcentration = particle.Y()[depositionSpeciesIndex_]*particle.pc()*molecularWeightMixture/(Ru_*particle.T()*molecularWeightDepositingSpecies); // [kmol/m^3]

                const scalar Rsurfrxn             = 2.2*surfaceReactionEfficiency_*gasPhaseConcentration*Foam::sqrt((M_PI*kb_*particle.T())/(2*monomerMassDepositingSpecies))*Foam::pow(d + dc_[section], 2.0); //
                [kmol/s] per particle

                const scalar dmfm                 = Rsurfrxn*molecularWeightDepositingSpecies*deltaT;                        // [kg] = [kmol/s] [kg/kmol] [s] per particle

                // calculation of the volumetric growth via the continuum collision kernel approach
                const scalar sigma                = 0.5*(3.667e-10 + d);
                const scalar eps                  = Foam::sqrt(190.0*97); // 190 for CO2 due to the similarity to SiO2, 97 due to the strong presence of N2
                const scalar keps                 = kb_/eps;
                const scalar Tstar                = particle.T()*keps;
                const scalar omega                = 1.06036/Foam::pow(Tstar, 0.15610) + 0.19300/Foam::exp(0.47635*Tstar) + 1.03587/Foam::exp(1.52995*Tstar) + 1.76474/Foam::exp(3.89411*Tstar);
                const scalar D                    = 1.883e-26*Foam::pow(particle.T(), 1.5)/(particle.pc()*Foam::pow(sigma, 2.0)*omega)*Foam::sqrt(1.0/molecularWeightDepositingSpecies +
                1.0/molecularWeightMixture); // [m^2/s]
                const scalar aggSurf              = a_[0]*(section+1);

                const scalar dmco                 = 2*D*particle.Y()[depositionSpeciesIndex_]/d_[section]*aggSurf*solidDensity_*deltaT; // [kg] = [m^2/s] [-] [m^2] [s] [kg/m^3] / [m]

                // calculation of the volumetric growth due to the harmonic mean
                const scalar dm                   = (dmfm*dmco)/(dmfm + dmco + SMALL);                                       // [kg]
```

Implementation of the synthesis model in mmcFoam

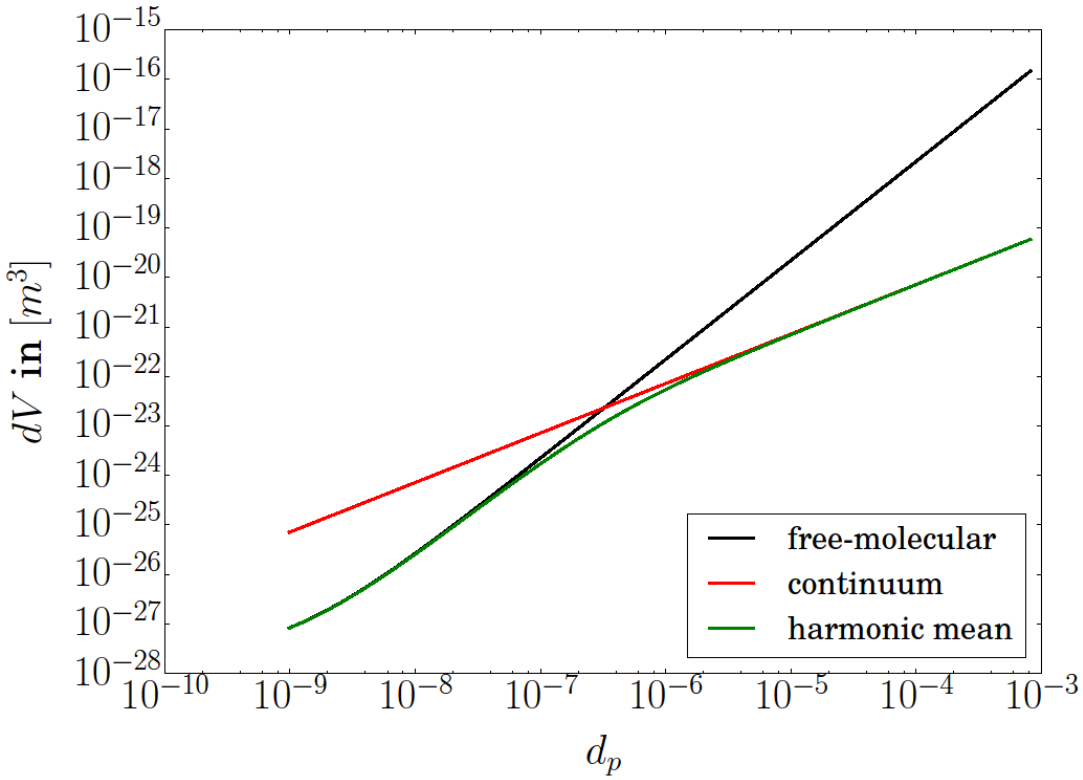
Surface growth of agglomerates

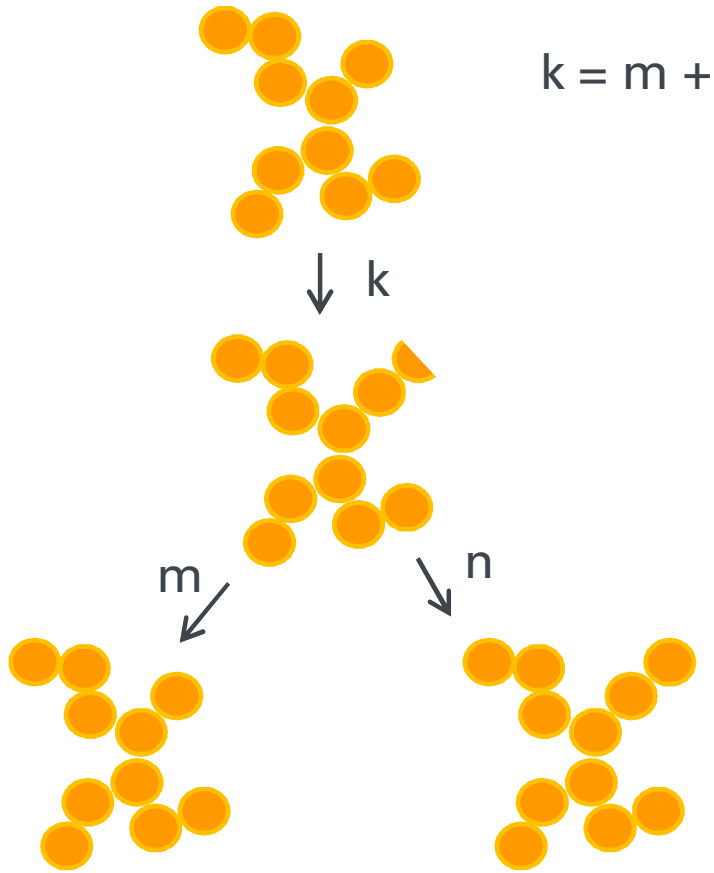
```
            scalar depositedMassFraction = dm*particle.sections()[section]/densityMixture;                    // [-] = [kg] [#/m^3] / [kg/m^3]

            if (depositedMassFraction > particle.Y()[depositionSpeciesIndex_])
            {
                depositedMassFraction = particle.Y()[depositionSpeciesIndex_];
            }

            const scalar dV = dm/solidDensity_;                                                               // [m^3] = [-] [kg] / [kg/m^3]

            // distribution of number density
            scalar targetVolume = v_[section] + dV;                                                           // [m^3] = [m^3] + [m^3]

            if (targetVolume > v_[particle.nSections()-1])
            {
                targetVolume = v_[particle.nSections()-1];
            }

            label targetSectionL = particle.nSections()-2;
            label targetSectionU = particle.nSections()-1;

            for (label searchSection = 0; searchSection < particle.nSections()-2; searchSection++)
            {
                if ((targetVolume >= v_[searchSection]) && (targetVolume <= v_[searchSection+1]))
                {
                    targetSectionL = searchSection;
                    targetSectionU = searchSection + 1;
                }
            }

            const scalar xiU = (targetVolume-v_[targetSectionL])/(v_[targetSectionU] - v_[targetSectionL]);
            const scalar xiL = 1 - xiU;

            const scalar nParticlesToEnterL = xiL*particle.sections()[section];
            const scalar nParticlesToEnterU = xiU*particle.sections()[section];
            const scalar nParticlesToLeave  = nParticlesToEnterL + nParticlesToEnterU;

            sectionChange[targetSectionL] += nParticlesToEnterL;
            sectionChange[targetSectionU] += nParticlesToEnterU;
            sectionChange[section]        -= nParticlesToLeave;

            particle.Y()[depositionSpeciesIndex_]    = max(particle.Y()[depositionSpeciesIndex_] - depositedMassFraction, 0.0);
            particle.Y()[solidSpeciesIndex_]         = min(particle.Y()[solidSpeciesIndex_]      + depositedMassFraction, 1.0);
        }

        forAll (particle.sections(), section)
        {
            particle.sections()[section] += sectionChange[section];
        }
    }
  }
}
```

Implementation of the synthesis model in mmcFoam

Surface growth of agglomerates

Since both, primary particle diameter and fractal dimension are no **free** dimensions in our model, the volumetric growth is an integral growth. To ensure conservation of primary particle number and mass the grown particles are distributed in the adjacent size section.

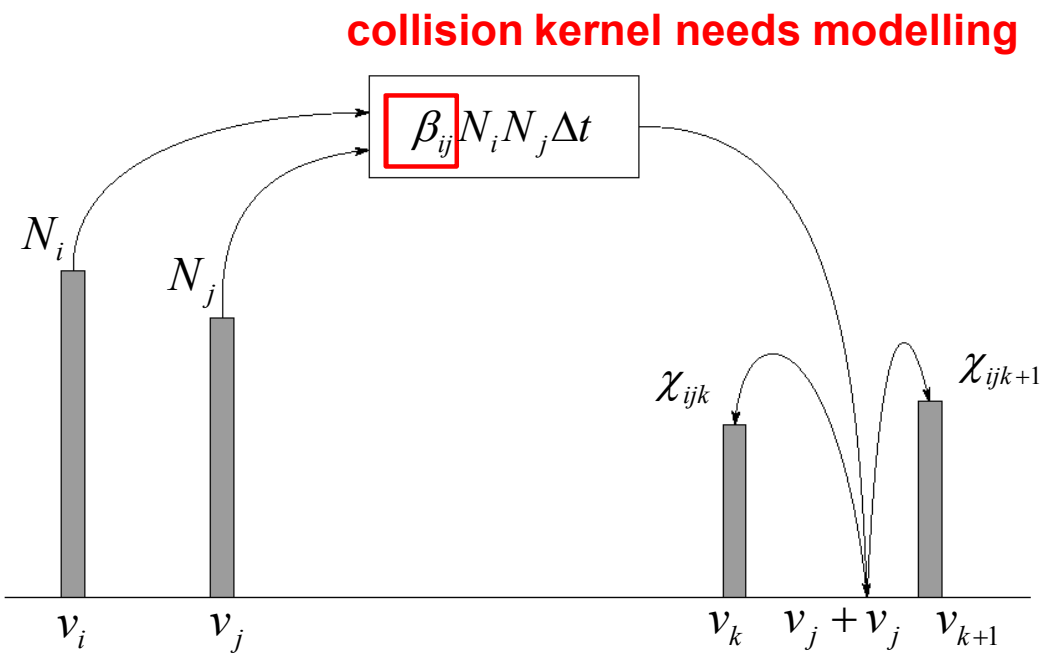




Implementation of the synthesis model in mmcFoam

Agglomeration

In our sectional approach two particles of volume v_i and v_j stick together and form a new particle with volume $v_i + v_j$. This particle is unlikely to perfectly match into an existing size section and is therefore distributed to the two adjacent size sections k and k+1 as depicted here:



$$\chi_{ijk} = \begin{cases} \dfrac{v_{k+1} - (v_i + v_j)}{v_{k+1} - v_k}; & \text{if } v_k \le v_i + v_j \le v_{k+1} \\ \dfrac{(v_i + v_j) - v_{k-1}}{v_k - v_{k-1}}; & \text{if } v_{k-1} \le v_i + v_j \le v_k \\ 0; & \text{otherwise} \end{cases}$$

Implementation of the synthesis model in mmcFoam

Agglomeration

There is a manifold of existing collision kernels available in the literature. For the free molecular regime (Kn > 1) and for spherical particles (D_f = 3) the following collision kernel is usually used (Friedlander book, p. 192) :

$$\beta_{ij} = \left(\frac{3}{4\pi}\right)^{1/6}\left(\frac{6k_BT}{\rho_s}\right)^{1/2}\left(\frac{1}{v_i}+\frac{1}{v_j}\right)^{1/2}\left(v_i^{1/3}+v_j^{1/3}\right)^2$$

For the continuum regime (Kn < 1) the following collision kernel in proposed:

$$\beta_{ij} = \frac{2k_BT}{3\mu}\left(\frac{1}{v_i^{1/3}}+\frac{1}{v_j^{1/3}}\right)\left(v_i^{1/3}+v_j^{1/3}\right)$$

But these are only valid for spherical particles and we need a collision kernel for agglomerates which shape can be described by the power law of agglomerates and is valid on both regimes.

Implementation of the synthesis model in mmcFoam

Agglomeration

For agglomerates which can be described by the power law of agglomerates Kruis et al. (2007) proposed an enhanced version of the Fuchs-Sutugin collision kernel presented in Seinfield (1986), which is valid for both, the free molecular and continuum regime, respectively.

$$\beta_{ij} = 2\pi(D_i + D_j)\frac{d_{c,i} + d_{c,j}}{\dfrac{d_{c,i} + d_{c,j}}{d_{c,i} + d_{c,j} + 2\sqrt{g_i^2 + g_j^2}} + \dfrac{8(D_i + D_j)}{(d_{c,i} + d_{c,j})\sqrt{c_i^2 + c_j^2}}}$$

$$d_c = d_p N^{1/D_f} \qquad Kn = \frac{2\lambda}{d_c} \qquad D = \frac{k_B T}{3\pi\mu d_c}\frac{5 + 4Kn + 6Kn^2 + 18Kn^3}{5 - Kn + (8+\pi)Kn^2}$$

$$m_p = v\rho_p \qquad c = \sqrt{\frac{8k_B T}{\pi m_p}} \qquad l = \frac{8D}{\pi c} \qquad g = (d_c + l)^3 - \frac{(d_c^2 + l^2)^{1.5}}{3d_c l} - d_c$$

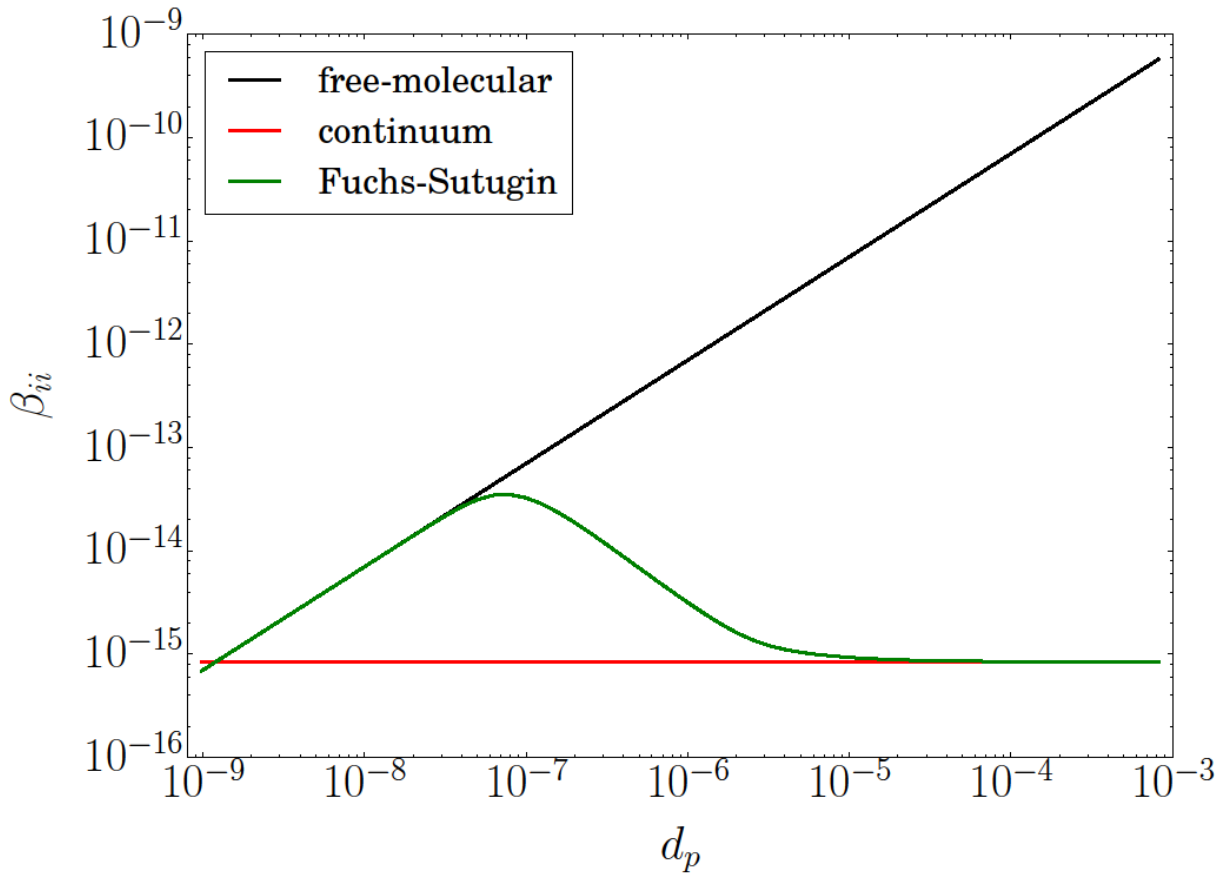
Implementation of the synthesis model in mmcFoam

Agglomeration

```
if (coagulationModel_ == "FuchsSutuginCollisionKernel")
{
    scalar A = 1.966e-06;
    scalar B = 147.47;

    const scalar& mu = A*Foam::pow(particle.T(), 1.5)/(particle.T() + B);

    const scalar& lambda = mu/particle.pc()*sqrt(M_PI*Ru_*particle.T()/(2.0*0.04));

    for (label i=0; i <= nSec-1; i++)
    {
        for(label j=0; j <= nSec-1; j++)
        {
            const scalar dci = dc_[i];
            const scalar dcj = dc_[j];

            const scalar Kn1 = (2.0*lambda)/dci;
            const scalar Kn2 = (2.0*lambda)/dcj;

            const scalar D1 = (kb_*particle.T())/(3.0*M_PI*mu*dci)*((5.0 + 4.0*Kn1 + 6.0*Kn1*Kn1 + 18.0*Kn1*Kn1*Kn1)/(5.0 - Kn1 + (8.0 + M_PI)*Kn1*Kn1));
            const scalar D2 = (kb_*particle.T())/(3.0*M_PI*mu*dcj)*((5.0 + 4.0*Kn2 + 6.0*Kn2*Kn2 + 18.0*Kn2*Kn2*Kn2)/(5.0 - Kn2 + (8.0 + M_PI)*Kn2*Kn2));

            const scalar mp1 = v_[i]*solidDensity_;
            const scalar mp2 = v_[j]*solidDensity_;

            const scalar c1 = sqrt((8.0*kb_*particle.T())/(M_PI*mp1));
            const scalar c2 = sqrt((8.0*kb_*particle.T())/(M_PI*mp2));

            const scalar l1 = (8.0*D1)/(M_PI*c1);
            const scalar l2 = (8.0*D2)/(M_PI*c2);

            const scalar g1 = (Foam::pow((dci + l1), 3) - Foam::pow((dci*dci + l1*l1), 1.5))/(3.0*dci*l1) - dci;
            const scalar g2 = (Foam::pow((dcj + l2), 3) - Foam::pow((dcj*dcj + l2*l2), 1.5))/(3.0*dcj*l2) - dcj;

            beta_[i*nSec+j] = 2.0*M_PI*(D1 + D2)*(dci + dcj)/((dci + dcj)/(dci + dcj + 2.0*sqrt(g1*g1 + g2*g2)) + (8.0*(D1 + D2))/(sqrt(c1*c1 + c2*c2)*(dci + dcj)));
        }
    }
}
```

Implementation of the synthesis model in mmcFoam

Agglomeration



The Fuchs-Sutugin collision kernel enables the blending between the free-molecular and continuum regimes for fractal structures following the power law of agglomerates.

The kernel was calculated for a solid density of 2196 kg/m³ and a temperature of T = 1000 K.

Implementation of the synthesis model in mmcFoam

Agglomeration

After the collision kernel and the size-splitting operator is computed the actual change in the number densities is computed:

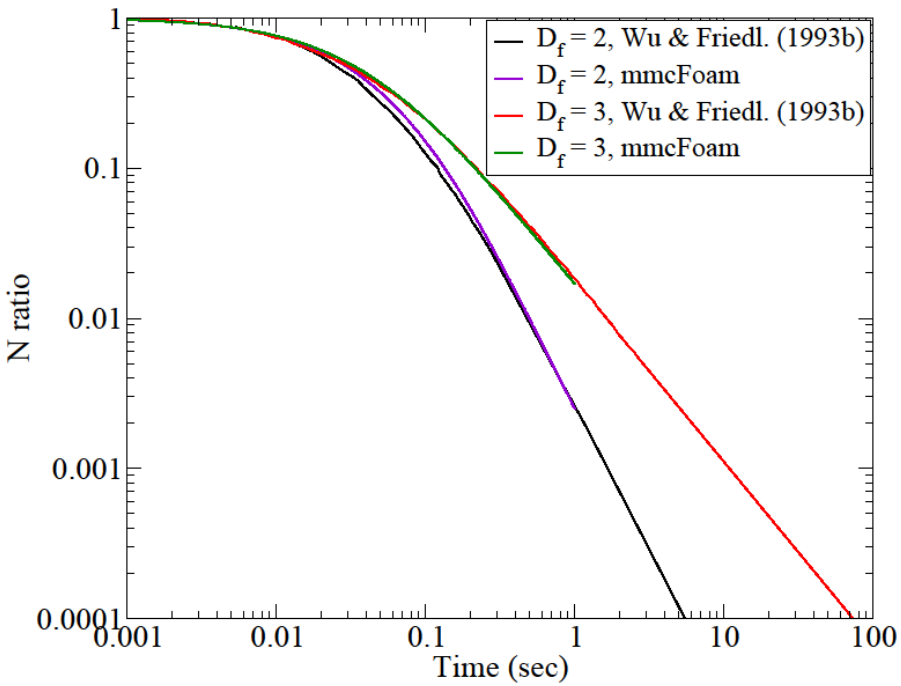
$$\dot{s}_{k,agg} = \frac{1}{2}\sum_{i=1}\sum_{j=1}\chi_{ijk}\beta_{ij}N_iN_j - N_k\sum_{i=1}\beta_{ik}N_i$$

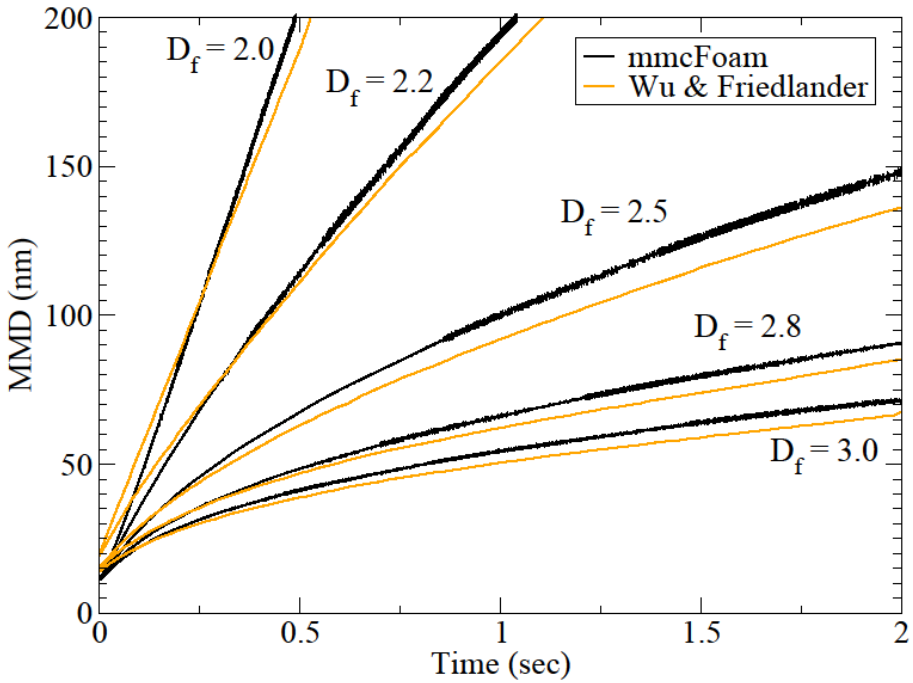
```
// calculating the gain and loss terms due to coagulation.
for (label k=0; k <= nSec-1; k++)
{
    scalar add = 0.0; // addition term when i and j collide to form a k sized particle.
    scalar sub = 0.0; // subtraction term when k collides with any other particle.

    for (label i=0; i <= nSec-1; i++)
    {
        sub += beta_[k*nSec+i]*particle.sections()[i];

        for (label j=0; j<=k; j++)
        {
            add += X_[i*nSecSqr+j*nSec+k]*beta_[i*nSec+j]*particle.sections()[i]*particle.sections()[j];
        }
    }

    particle.sections()[k] += deltaT*(0.5*add - particle.sections()[k]*sub);
}
```

Implementation of the synthesis model in mmcFoam

Agglomeration

Validation of the particle agglomeration model can be done by comparing our simulation results with results obtained with direct Monte-Carlo methods.





M. K. Wu and S. K. Friedlander (1993) Enhanced power law agglomerate growth in the free molecular regime J. Aerosol Sci., Vol. 24, No. 3, pp. 273-282.

Vielen Dank!

Gregor Neuber

E-Mail gregor.neuber@web.de
Telefon +49 (0) 711 685-653 09
www. itv.uni-stuttgart.de

Universität Stuttgart
Institute of Combustion Technology
Herdweg 51
70174 Stuttgart
Deutschland

References

Suh, S.-M., Zachariah, M.R. and Girshik, S.L. “Modeling particle formation during low-pressure silane oxidation: Detailed chemical kinetics and aerosol dynamics”. J. Vac. Sci. Technol. A 19:940–951. 2001.

Tamura, M., Berg, P.A., Harrington, J.E., Luque, J., Jeffries, J.B., Smith, J.B. and Crosley D.R. “Collisional Quenching of CH(A), OH(A), and NO(A) in Low Pressure Hydrocarbon Flames”. Combustion and Flame 114:502–514. 1998.

Friedlander, S.K. “Smoke, Dust, and Haze: Fundamentals of Aerosol Dynamics”. Second Edition. Oxford University Press. 2000.

Prakash, A., Bapat, A.P., and Zachariah, M.R. “A Simple Numerical Algorithm and Software for Solution of Nucleation, Surface Growth, and Coagulation Problems”. Aerosol Science and Technology, 37:892–898, 2003.

Kronenburg, A., Bilger, A.W. and Kent, J.H. “Modeling Soot Formation in Turbulent Methane–Air Jet Diffusion Flames”. Combustion and Flame 121:24–40. 2000.

Sorensen, C.M. “The Mobility of Fractal Aggregates: A Review”. Aero. Science and Technology 45:765–779. 2011.

Kruis, F. E., Kusters, K. A., Scarlett, B., Pratsinis, S. E., Aerosol Sci. Technol. 19 (1993) 514–526.

Wu, M. K., Friedlander, S. K. “Enhanced power law agglomerate growth in the free molecular regime”. J. Aerosol Sci. 24:3, 273-282, 1993.